

Model-Agnostic and Language-Agnostic Voice Pipeline Improvement for the Agriculture Domain

Aakash Singh^{*} Lakshmi Pedapudi[†] Chandrashekar M S[‡] Sanyam Singh[§]
 Naga Ganesh[¶] Vineet Singh^{||}
 Digital Green

Abstract

FarmerChat is Digital Green’s AI-powered agricultural advisory assistant for smallholder farmers, who reach it in their own language through text, voice, or photographs, whichever is most convenient; it has answered millions of questions for hundreds of thousands of farmers across several countries [1]. Voice is a critical channel for this population, because many users have limited literacy or limited comfort typing in their own script, so a large share of questions arrive as field recordings made on low-end phones, often over farm machinery such as a tractor or pump, a television or radio playing nearby, or a second person helping them use the app. General-purpose automatic speech recognition (ASR) transcribes this audio poorly. Field noise, extra speakers, and dense agricultural vocabulary each degrade the transcript, and the errors that matter most fall on the crop, pest, chemical and quantity terms that carry the meaning of the question.

This paper reports how Digital Green is addressing these failures in its own voice advisory service. We build a modular pipeline that wraps an unmodified ASR model and targets each failure mode with a dedicated stage: audio analysis with gated enhancement, speaker diarization with target-speaker selection, the ASR call itself, domain-aware correction against a weighted agricultural lexicon, and a quality gate that catches failed transcripts before they reach the downstream advisory model, whose design is specified here and whose measurement is future work. The pipeline improves transcription without fine-tuning the ASR model, replacing the provider, or adding a large agentic system.

We evaluate it on human-annotated FarmerChat recordings in Hindi, Telugu and Odia, scoring each stage with word error rate and a domain-weighted error rate that penalizes agricultural-term mistakes more heavily, rather than word error rate alone. The gain concentrates on multi-speaker audio: once the diarizer isolates the farmer’s own speech, competing voices stop entering the transcript, and this effect holds across ASR models from different model families, which is what makes the improvement model-agnostic. Because a general-purpose diarizer does not transfer cleanly across these languages, the segmenter is the one component we fine-tune; every other stage uses an off-the-shelf model behind a common interface. The design targets incremental deployment, with each stage configurable and independently replaceable, and enhancement and correction applied conditionally, where the evidence supports them.

On the full corpus the pipeline lowers word error rate by a relative 16% to 23% on three cloud ASR models and 5% on an on-device model, and by 32% to 42% on multi-speaker audio, 16% on the on-device model. Every drop is statistically significant on all four. Section 7 reports the stage-by-stage results.

1 Introduction

1.1 Motivation

A farmer standing in a field can talk. Typing is much harder. So voice is the natural way in, and the audio that arrives is nothing like clean benchmark speech:

- it is recorded in the open air on a cheap phone microphone, with other people talking, farm machinery running, a television or radio playing, and wind;
- more than one person speaks, because an extension worker or a relative often helps the farmer use the app and talks while doing it;
- speakers mix languages, and they use farming words (crops, pests, chemicals, doses, units) that general ASR models rarely hear;

^{*}aakash@digitalgreen.org

[†]Corresponding author: laxmigenius@gmail.com

[‡]chandrashekar@digitalgreen.org

[§]sanyam@digitalgreen.org

[¶]naga@digitalgreen.org

^{||}vineet.vinsing@gmail.com

- nothing warns anyone when the transcript is wrong, so the advisory model answers a question the farmer never asked.

Not every word costs the same. Getting a function word wrong is harmless. Getting the crop, the disease, the pest, the chemical, the quantity or the place wrong changes the question, and a changed question can send a farmer to the wrong treatment. Table 1 lists the confusions we see most often in our own recordings. Plain word error rate hides them, because it charges the same for every word.

1.2 Problem Statement

The production ASR model is tuned for general transcription quality. It is not tuned for getting farm queries right. This paper asks a narrow question: can a few cheap processing stages, plus repair of farming words, make an *existing* ASR model more useful without

- fine-tuning the ASR model,
- changing the ASR model provider,
- building a large agent or language-model system,
- or adding much compute or serving infrastructure?

The pipeline measured here meets all four conditions, and none of its stages makes a language-model call of its own. A small contextual pass over the cases the fixed rules cannot settle safely is named as future work (§9), not as part of the measured system.

1.3 Research Questions

- **RQ1.** How much does cleaning the audio help, and for which ASR models? (§5.1, §7.3)
- **RQ2.** How much does splitting the speakers and keeping the farmer help when more than one person talks? (§3.3, §5.2, §7.1)
- **RQ3.** Can farming words be repaired without touching the ASR model? (§5.4, §7.5)
- **RQ4.** What does all this cost in money, latency and deployment effort? (§4.3, §7.6)

Whether the final answer to the farmer is good is outside the scope of this paper, because it depends on the model that reads the transcript (§6.5). The closest measurable thing is how many farming terms survive, and that is reported at every stage.

1.4 Contributions

1. **A voice pipeline that does not depend on the ASR model or the language.** Five stages that can each be replaced, one of them the ASR model itself, left untouched. Only the diarization segmenter is fine-tuned (§4, §5).
2. **A rule-based repair stage, and the word list behind it.** About 18,000 farming terms mined from roughly 100,000 farmer queries. Every sound-alike pair is labelled as either the same word twice or two different words, so the repair stage knows which pairs need context and must be left alone. The build runs again per language (Appendix A).
3. **A measurement of every stage** on four ASR models, cloud and on-device, in three languages, scored against the farmer’s own words. We report word error rate, a weighted error rate that charges more for farming words, a farming-term rate, and paired bootstrap intervals (§6, §7). The full pipeline cuts word error rate by 16% to 23% on the three cloud ASR models, 5% on the on-device model, and 16% to 42% where more than one person speaks, the largest gains landing where there is most to fix.
4. **Evidence for when each conditional stage pays, and the gating that follows from it.** Cleaning helps a generative model on its noisy tail and is not worth running on the others, so it is switched on per model and per noise tier. Repair is restricted to words the mining corpus has never seen, which is what lets it recover garbled farming terms while leaving valid words alone (§7.3, §7.5).
5. **Open-source releases.** The evaluation set, with both the farmer reference and the full per-speaker transcription for every clip,¹ the word list with its labelled pairs,² the fine-tuned diarization checkpoint,³ the pipeline and evaluation code,⁴ and a public demonstration⁵ (§11).

¹Evaluation set (CC-BY-4.0): <https://huggingface.co/datasets/DigiGreen/agri-voice-eval> [2]

²Farming word list (CC-BY-4.0): <https://huggingface.co/datasets/DigiGreen/agri-lexicon-hindi> [3]

³Diarization segmenter (MIT): <https://huggingface.co/DigiGreen/pyannote-segmentation-agri-indic> [4]

⁴Pipeline and evaluation code: <https://github.com/aakashdg/agri-voice-pipeline> [5]

⁵Demonstration: <https://huggingface.co/spaces/DigiGreen/farmerchat-voice-pipeline-demo> [6]

2 Existing System and Baselines

2.1 Production Voice Pipeline

Figure 1 shows what happens to a farmer’s recording today. FarmerChat records the question on the farmer’s or the extension worker’s phone. One ASR model call follows, from a different provider per country and language: Google ASR v2, Navana Tech or Sarvam AI. The provider, not Digital Green, then accepts or rejects the clip on length and noise. Whatever survives goes downstream, where the rest of the advisory pipeline treats the transcript as if the farmer had typed it. Nothing on that path looks after farming words, several speakers, or a transcript that is simply wrong.

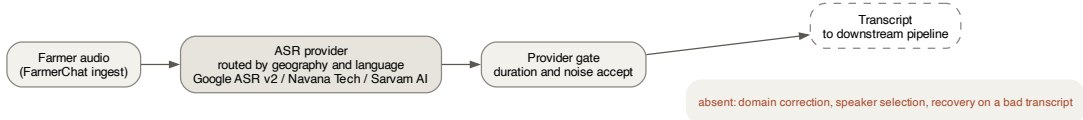


Figure 1: The pipeline in production today. Red marks the three things it does not do.

2.2 Baseline Systems

Four comparison points frame this work. Every result is measured against the first; the other three set the context, and §6.4 says which of them was run.

- **Baseline 0, vanilla ASR model.** Raw audio in, transcript out, no gate and no repair. The floor.
- **Baseline 1, production pipeline.** The current FarmerChat path of Figure 1. It serves live traffic with no paired reference, so it sets the deployment context the pipeline has to fit.
- **Baseline 2, native-audio model.** One multimodal model answers straight from the audio, with no transcript step. It is the costlier alternative, and its answers have to be weighed against its cost, its latency, and the loss of a transcript anyone can score. That last point is the reason every configuration here produces a transcript, even when the ASR model is itself a multimodal model (§4.3).
- **Baseline 3, the pipeline’s own ladder.** The stages of §5 switched on one after another, every configuration scored against the same reference (§6.4). This is what shows what each stage adds.

2.3 Related Work

Our earlier benchmark [7] tested ten ASR models on a multi-language corpus of farmer recordings (Table 6). It introduced the Agriculture Weighted Word Error Rate (AWWER), and it showed that picking the main speaker after diarization is the single most useful thing you can do to multi-speaker audio. This paper builds the pipeline that benchmark asked for and measures it as a whole. The advisory model that reads these transcripts, and the way its answers are checked fact by fact, are described in Singh et al. [1].

General speech recognition has improved quickly, both in large multilingual models such as Whisper [8] and Meta’s Massively Multilingual Speech project [9] and in commercial cloud services. Indian-language coverage has grown through national and academic work: Bhashini [10], Vaani [11], work toward recognition for the next billion users [12], Vakyansh [13] and SPRING-INX [14]. Most of that training audio is read or broadcast speech in fairly clean conditions, which is not how farmers speak, and coverage across our three languages is still uneven [7, Table II]. That is why we treat the ASR model as a part to be swapped rather than a thing to be fixed.

The processing stages use established tools: DeepFilterNet for cleaning [15]; pyannote [16, 17], ECAPA-TDNN embeddings [18], NeMo MSDD and Sortformer [19, 20] and DiariZen [21] for diarization; Silero for voice activity detection [22]. Diarization is scored with DER and with concatenated minimum-permutation WER (cpWER) as defined for CHiME-6 [23]. For repairing domain words after recognition, Garg et al. [24] combine spelling distance, sound similarity and context to keep only plausible candidates, and Ma et al. [25] restrict a corrector to a fixed candidate list instead of letting it rewrite freely. Our repair stage keeps the first two signals and the fixed list, and leaves anything that needs context alone (§5.4.2).

3 Failure Analysis of the Existing Pipeline

This section is about where Baseline 0 and Baseline 1 break. The evidence comes from the published benchmark corpus (Table 6; recorded June 2024 to February 2025) [7], the human quality-checked corpus, and the reference

annotated corpus. Pilot values carry the marker ^P. Values a language-model classifier produced, rather than code, carry ^L. Each subsection below takes one failure and the evidence for it.

3.1 Domain-Specific Recognition Errors

- Farming words get swapped for something that sounds like them, or dropped outright. Crop names and fertilizer or chemical names are the biggest group in all three languages. The most common pairs in our own recordings are in Table 1.
- Under AWWER, which charges more for farming words, the errors sit on exactly those words [7, §4.4.4].
- Sound distance does not tell you which pairs are dangerous. A word swapped for a sound-alike with a different meaning nearly always changes the question. The same word spelled two ways nearly never does. Which of the two a pair is has to be decided pair by pair, and that decision is what the repair stage is built on (Table 5).

Table 1: The farming confusions we see most often, and what each one does to the question.

Reference term	ASR model output	What changes	Count
दावा (<i>davā</i>), medicine or pesticide	दावा (<i>dāvā</i>)	A treatment becomes a claim	335
खाद (<i>khād</i>), fertilizer	खाद्य (<i>khādya</i>)	Fertilizer becomes food	194
बीज (<i>bīj</i>), seed	बीच (<i>bīch</i>)	Seed becomes in-between	153
पत्ता (<i>pattā</i>), leaf	पता (<i>patā</i>)	A leaf becomes an address	144
रोपने (<i>ropne</i>), to transplant	रोकने (<i>rokne</i>)	Plant it becomes stop it	88
गेहूँ (<i>gehūn</i>), wheat	गेम (<i>gem</i>)	A crop becomes a non-word	81
मूँग (<i>mūng</i>), green gram	मुंह (<i>munh</i>)	A crop becomes a body part	74

3.2 Field Recording Conditions

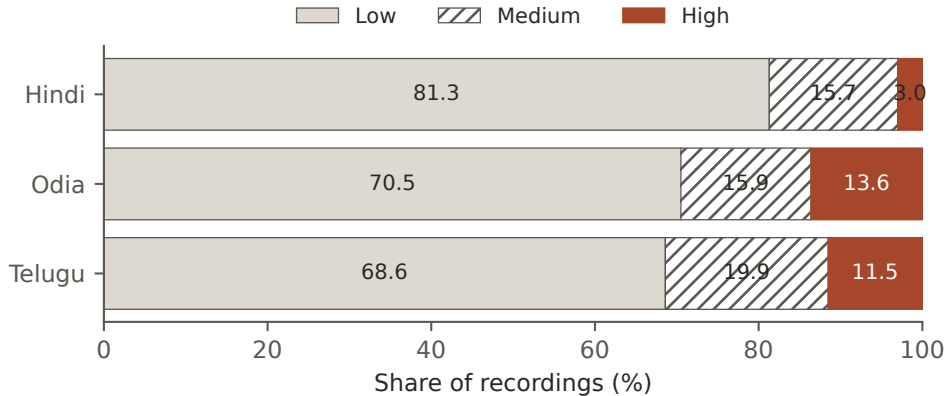


Figure 2: Noise level by language, as a share of recordings [7, Table I].

- Recordings are made outdoors on cheap phone microphones. Tractors and irrigation pumps run nearby. A television or radio plays. Wind hits the microphone, and the open field adds echo. Other people talking is the most common problem of all, in every one of the three languages [7].
- This is a well known weak spot. ASR models trained on clean read speech have always struggled with noisy, distant, echoing audio, which is the whole point of the CHiME challenge series [23].
- The three languages are not affected equally. High-noise audio is about four times as common in Odia as in Hindi, with Telugu in between (Figure 2). So a cleaning stage has to react to the clip in front of it instead of treating every clip the same (§5.1).
- Noise costs meaning, not just words. In the reference annotated corpus, the noisier the clip, the more often the farmer’s point is lost altogether (Table 2)^L.

Table 2: How often the farmer’s point survives by noise tier, one cloud ASR model against the human reference. Language-model judged^L.

Noise tier	Intent preserved	Partly preserved	Intent lost
Low	83%	11%	6%
Medium	82%	11%	7%
High	78%	12%	10%

3.3 Multiple Speakers

A field visit is rarely one person talking. An extension worker or a family member often works the app for the farmer and talks while doing it, and other people are usually around.

- About one clip in five in the human quality-checked corpus has more than one speaker, and about 4% have three or more.
- Two people talking at once is one of the hardest things in recognition. When the second voice is nearly as loud as the farmer's, an ASR model that returns one transcript writes down both and cannot say which words were the question. This is why the CHiME series added multi-speaker tracks [23].
- Multi-speaker clips are harder for every diarizer we tested, open or cloud, which is why one of them was fine-tuned (§5.2).
- Picking out the main speaker before transcription is the largest single gain available. Our earlier benchmark showed it across ten ASR models and three languages: split the recording by speaker, score only the main speaker's turns, and word error rate on multi-speaker audio drops a long way. The size of the drop follows how often that ASR model meets a second speaker [7, §4.6, Table VII]. This pipeline turns that result into a stage, and §7.1 measures what it is worth on our corpus with four ASR models in the identical pipeline.
- Without selection, the farmer's question sits buried inside the extension worker's speech.

3.4 Lack of Error Recovery

The production path has no idea whether a transcript is any good. A garbled transcript goes to the advisory model exactly as a clean one does. There is no accept step, no reject step and no reroute (Figure 1). A language model answers fluently from whatever text it gets, so a wrong transcript does not look wrong: the farmer gets a confident answer to a question nobody asked, with nothing to warn that the input was broken. That silence makes every other failure worse, because a noise or speaker error that gets this far is acted on instead of caught. How often a question reaches the advisory model with its point already lost is in Table 2^L.

3.5 Evaluation Blind Spot

The last failure is in how we measure, not in the audio.

- Word-level metrics treat every substitution alike, so a ruined crop name looks the same as a ruined function word. This weakness of string-edit metrics is old news [26].
- The ASR model with the best WER is not the one that gets farming words right. On the published benchmark in Hindi, one system is first on WER and fifth on AWWER, while another with main-speaker selection is third on WER and first on AWWER [7, §4.5]. A metric that cannot tell a usable transcript from a dangerous one cannot steer the system.
- Not every mangled word matters equally. Some change the question and some are still clear from context, and you cannot tell which from sound alone (Table 5).
- That is why the evaluation reports a weighted rate and a farming-term rate next to WER (§6.5) instead of WER by itself.

3.6 Reference Annotation Quality

The last failure is in the reference itself. A person transcribing field audio hears through the mess without meaning to, and a machine does not, so a single-line reference collected without an explicit protocol records something slightly different from what was said. Two effects matter for scoring, and the protocol below answers both.

- **People write the word they know was meant.** An annotator who knows the crop hears the intended word and writes that, not the sound the farmer actually produced. Our own sheet shows the reflex directly: on many clips the annotator wrote the standard spelling in braces next to what was said (Table 3). That is useful only because it was written down. Without that convention the reference records the intended word rather than the spoken one.
- **One line of text hides several speakers.** A reference kept as a single sentence per clip merges whoever was talking. On a sizeable share of the multi-speaker clips, the single-line reference carries a second speaker's words as well as the farmer's. Score against that and a ASR model earns credit for transcribing a bystander, which is the opposite of what the service needs.
- **Fields that contradict each other.** A handful of clips carry a segment that ends before it starts, both speaker-count ticks at once, or an empty farming-word field that could mean either no farming word or no entry. Each is small, and together they are why the protocol fixes the schema as well as the wording.

A reference of this kind needs an explicit protocol. We wrote one that asks for each speaker's turns with their own timestamps, a label naming the speaker who asked the farming question, audio issues from a fixed

Table 3: An audit of the annotation sheet, 825 clips, 275 per language. Each row counts clips.

What the audit looked for	Clips	Why it matters
Standard spelling supplied in braces beside what was said	183	The annotator heard the intended word, not the spoken one
Single-line reference carries another speaker’s words, of 157 multi-speaker clips	47	Scoring against it rewards transcribing a bystander
No farming word listed	78	Really absent, or an unfilled field; the sheet cannot say
Segment ends before it starts	6	Breaks any script that trusts the timestamps
Ticked single-speaker and multiple-speaker at once	3	The audio-issue list is not mutually exclusive
Speaker count disagrees with the turn labels used	1	Two fields that should agree, do not

list, and the farming words with their categories. What was said gets written, and the standard form goes in braces beside it. We ran that protocol as a trial with the Indian Institute of Science, Bangalore, and §6.2 sets it out with a worked clip.

4 Design Principles and Architecture

4.1 Design Principles

The failures in §3 lead to five rules that shaped every choice here. Put together models that are each already good at one job. Keep every stage switchable. Judge the system by whether the farmer’s question survives, not by word-for-word match alone.

- **P1, use specialists.** The ASR model, the voice activity detector, the diarizer and the cleaner each do the one thing they are good at (§4.3).
- **P2, retrain as little as possible.** Improve the system around the ASR model. Fine-tune only where nothing off the shelf works, which here is one model, the diarization segmenter (§5.2).
- **P3, make every stage configurable.** Model choice, thresholds, confidence levels and accept or reject rules are all settings, not code. This is what supplies the missing recovery step of §3.4.
- **P4, score the question, not the string.** Check whether the farmer’s question came through, not only whether the words match (§3.5).
- **P5, count cost as well as accuracy.** Accuracy, money, latency and deployment effort together.

Replacing or fine-tuning the ASR model instead (against P2) buys accuracy but costs provider freedom, training and serving machinery, and a great deal of hand annotation that is itself unreliable on noisy multi-speaker audio (§4.3).

4.2 High-Level Architecture

Five modules make up the pipeline, and one of them is the ASR model call, which can be swapped (Figure 3). Each module can be replaced or switched off by itself, and the ladder of §6.4 switches them on one at a time. Table 4 says what each one does and which of them carries a trained weight. Only the recovery gate (M4) branches. Everything else is a straight line.

Table 4: What each module does. “Trained” means a model weight changed for this system.

Module	Function	Models	Trained	Section
M0	Measures audio quality, then cleans it only if needed	Signal statistics; DeepFilterNet3	No	§5.1
M1	Finds speech, splits it by speaker, keeps the farmer	Silero VAD; pyannote segmenter and clusterer	Segmenter only	§5.2
M2	Turns audio into text	Any provider behind one interface	No	§5.3
M3	Repairs garbled farming words	Labelled word list; sound matcher with safety gates	No	§5.4
M4	Judges the transcript and reroutes bad ones, design only	Confidence and consistency checks; routing rules	No	§5.5

4.3 Alternative Design Strategies

Two other routes were considered and dropped as the main plan.

- **Fine-tune the ASR model.** It would learn farm speech directly, so domain words may come out better. But ASR models are trained on clean audio, field audio has to be transcribed by hand first, and on noisy clips those hand transcripts are themselves unreliable, so the model learns from bad references. It also needs noisy training data, training and serving machinery, and it locks the system to one provider.

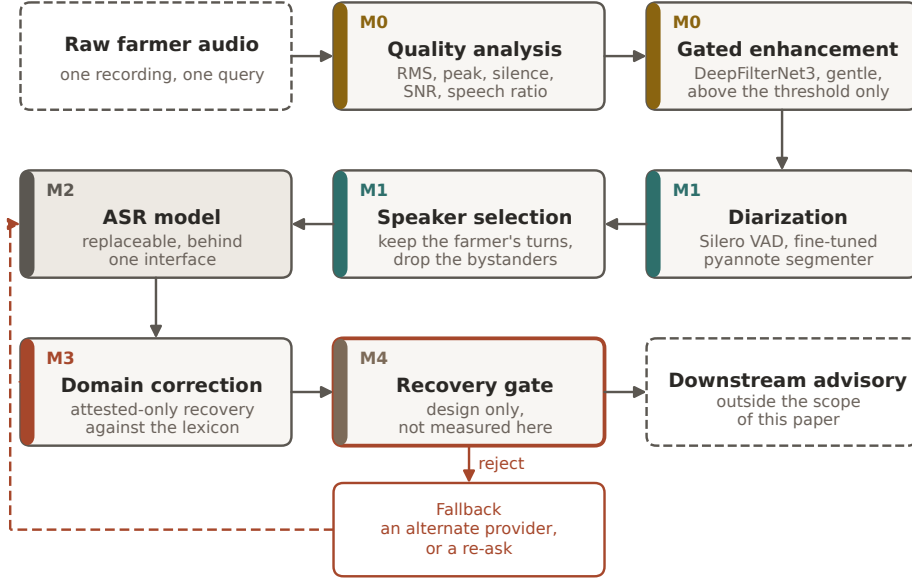


Figure 3: The proposed pipeline. Coloured edges mark the modules; M4 is design only.

- **Hand the whole job to a language model or agent.** Flexible reasoning, repair from context, simple to orchestrate. But the speech problems underneath, noise and several speakers, are still there. There is less control over the specialist models, repair quality rides on the prompt, and cost and latency per query go up and get harder to predict. The audio-in, answer-out variant leaves no transcript to score (Baseline 2, §2.2).
- **The modular pipeline, which is what we built.** One specialist per job, explicit control at every stage, parts that swap out, measurement per part, almost no new infrastructure. The price is more moving parts than a single model call, and correctness that depends on tuning each gate.

No stage of the pipeline reported here adds a language-model call of its own. Repair follows fixed rules, and the contextual pass that would settle the sound-alike cases is named as future work rather than built in (§5.4.2, §9). The native-audio point matters for measurement as much as for cost. A model that goes straight from audio to an answer can only be judged on the answer: WER, AWWER and cpWER stop being defined. Keeping a transcript in the middle is what lets every stage in this paper be measured on its own.

5 Pipeline Modules

5.1 M0: Audio Analysis and Enhancement

The stage looks at the audio first and cleans it only where cleaning is likely to help.

- **Quality detection.** RMS level, peak level, how much of the clip is silence, an estimate of signal against noise, and how much of the clip is speech. All of it is arithmetic on the waveform, computed before any model runs, so deciding whether to clean costs almost nothing. A single loudness threshold recovered almost none of the gain, so the gate uses all the features together. That gate is what a deployment runs. In the runs reported here the tier comes from the corpus’s own noise label^L, which keeps the measured effect of cleaning separate from the accuracy of the gate that selects it.
- **Enhancement.** DeepFilterNet3 [15], a small CPU-only cleaning network, run gently instead of at its default maximum. At maximum it over-processes the signal and strips cues the ASR model needs, so the attenuation is capped at 6 dB. On noisy clips a generative ASR model invents words that were never spoken, and gentle cleaning cuts that down. What the stage costs to run is in §7.6.
- **Adaptive gating.** Two ways to decide, both deployment settings. The safeguard always runs the cleaner and keeps its output only where it removed enough energy to matter. The compute saver lets the signal features decide whether to run the cleaner at all, which skips most traffic because most traffic is already clean. The operating point and the per-query cost are in §7.6.
- **Enhancement is model-dependent, so knowing when to apply it is the finding.** Suppression tuned to sound better to a human can remove information an ASR model relies on, so less measured noise does not mean fewer errors. The stage therefore fires only above the noise threshold and only for a model whose own evidence supports it, and that threshold is checked again whenever the ASR model changes. Cleaning everything would not have that property. §7.3 reports which of the four ASR models it pays on.
- **Why M0 stays in the pipeline.** It pays on the noisy tail for the ASR model it suits. And it also cleans

the audio the diarizer then has to split, so part of its value shows up inside M1’s result, which is why the stage is judged by noise tier rather than by a flat corpus number.

5.2 M1: Speaker Detection and Selection

Getting to the farmer takes three steps: find the speech, work out who spoke when, keep the farmer’s turns.

- **Voice activity detection.** Silero VAD [22], cross-checked against the diarizer’s own segmentation. Gating pyannote with Silero lowers DER at the strictest setting but leaves the ASR model with too few words, so the DER-minimizing setting is not the one we use (§7.4).
- **Diarizer comparison.** Nine systems, seven open and two cloud, scored with one harness, and the fine-tune of the next bullet makes ten. No stock system split Hindi, Telugu and Odia field audio cleanly, and most of the error is false alarm: speech heard in background noise. That is the error the fine-tune goes after, and §7.4 has the comparison.
- **The one fine-tune in this system.** pyannote’s segmenter [16, 17], trained on the project’s own field audio, because no off-the-shelf model gave the false-alarm profile selection needs. It is scored on a frozen held-out split it never trained on, against the same reference at the same collar (§7.4). Swapping the clusterer underneath it changes almost nothing, so the segmenter is the lever, not the clusterer. The checkpoint and its recipe are released [4].
- **Best-speaker selection.** The rule that ships uses loudness with a duration floor. Among speakers who hold the floor for at least $\max(1.0 \text{ s}, \frac{1}{4} \text{ of the longest speaker's total time})$, the loudest on average is taken to be the farmer, because the farmer holds the phone. Their turns are padded a little at each end, so the first and last words are not clipped, and joined for the ASR model. Selection reads whatever the cleaning gate passed on, so the ladder measures the two stages in the order a deployment runs them. There is no voice sample of the farmer to enrol, so target-speaker selection from an enrolment is out of scope.
- **Why loudness, and not duration alone.** A rule that picks whoever talks longest fails on the case that matters most: a bystander who simply holds the floor longer than the farmer. Loudness separates that case, because a bystander is usually further from the microphone. Four text-only rules were scored against the annotated main-speaker label, and the shipped rule end to end through the transcript it produces, both in §7.4.

5.3 M2: ASR model, Unchanged and Replaceable

- The ASR model is the one part the pipeline does not touch. It sits behind a shared interface as a block you can swap: audio in, transcript out. That is what makes the system model-agnostic.
- **No single ASR model is good at all three languages.** Ten providers have been benchmarked on this audio and their coverage is uneven, because a system that leads one of the three can fall apart in another [7, Table II]. So routing is per language rather than one model for everything, and three providers run in production today: Google ASR v2, Navana Tech and Sarvam AI.
- **ASR models in the staged evaluation.** Four systems from different families and deployment styles, every one run through every stage (§6.7). That is what the model-agnostic claim is measured on.

5.4 M3: Domain-Aware Correction

The repair stage fixes farming words the ASR model got wrong. It works against a fixed, checked word list instead of rewriting freely, and the stage makes no language-model call of its own.

The open question was whether to use a language model or fixed rules, and measurement settled it (§7.5). What shipped is neither of the two original options: one fixed-rule stage that aims at precision rather than coverage.

5.4.1 Lexicon Construction

The word list is a contribution in itself. Farming words are mined from the reference annotated corpus, cleaned up, grouped by how they sound, and then every sound-alike pair is labelled as either the same word twice or two different words (Figure 4). The full seven-step build is Appendix A, and Table 5 is the labelling the repair rule depends on. The list is released with its labelled pairs [3].

That labelling is what lets the stage tell a harmless spelling variant from a dangerous swap. Around दवा (*davā*) (medicine) the list absorbs दवाई (*davāī*) and दवाएँ (*davāēn*), which mean the same thing, but refuses दावा (*dāvā*) (claim) and दबा (*dabā*) (pressed), which only sound the same. पत्ता (*pattā*) (leaf) absorbs पत्ते (*patte*) but refuses पता (*patā*) (address) (Figure 5). A corrector that rewrote every near-homophone to its most common neighbour would turn medicine into claim and leaf into address. Those refusals are what stop it.

The different-meaning pairs are not spread evenly across the categories, and where they pile up is in §7.5.

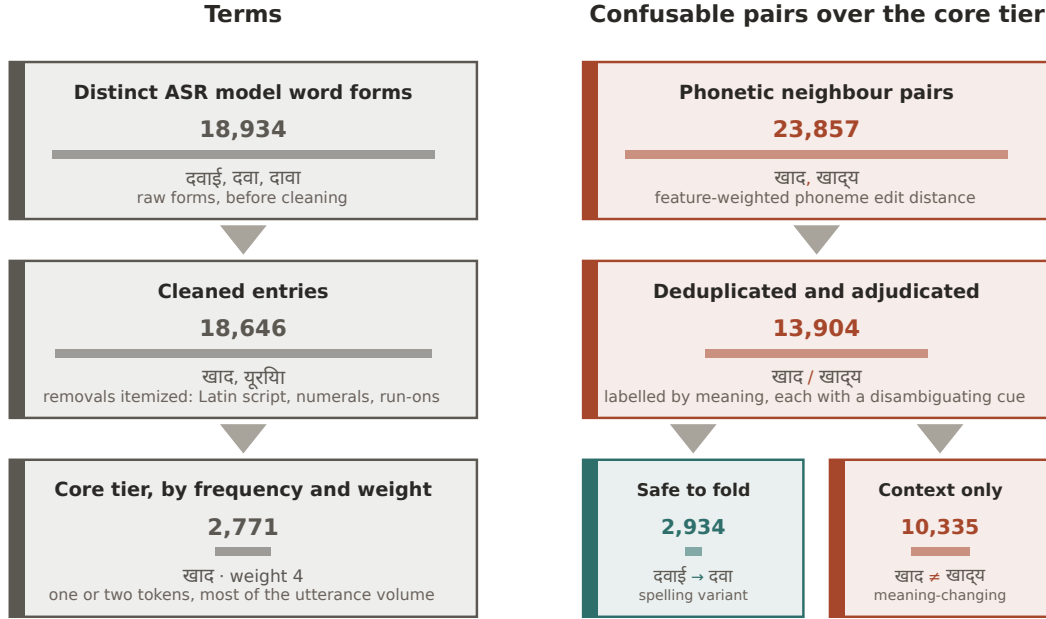


Figure 4: The word list build, with one worked example at each stage. Step detail in Appendix A.

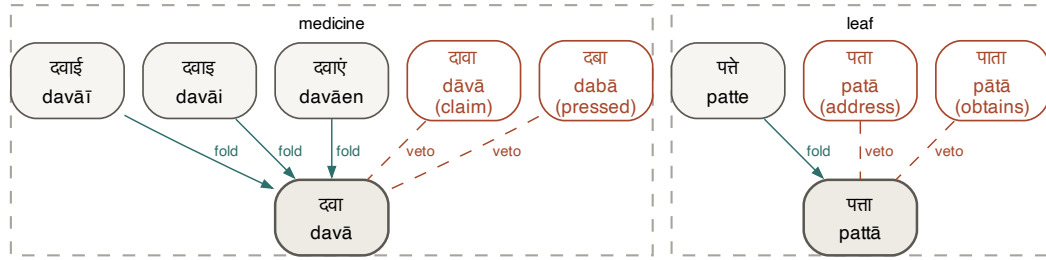


Figure 5: Two variant families. Folded variants collapse into the main word; refused ones stay separate.

Four weights are shared by the repair stage and by AWWER [7, §3.3.2]: weight 4 for core agriculture (crops, pests, practices), weight 3 for strongly related words (soil, weather, timing), weight 2 for indirectly related words (quantities, locations), and weight 1 for anything not in the list.

Table 5: How the sound-alike pairs were labelled, with an example of each^L.

Label	Pairs	Share	Conf.	Example	What happens to it
Different meaning	10,335	74%	0.80	खाद (<i>khād</i>) / खाद्य (<i>khādyā</i>)	Flagged with a cue, never rewritten
Same word, two spellings	2,287	16%	0.85	मकई (<i>makāi</i>) / मकाई (<i>makāi</i>)	May be folded to the main spelling
Same meaning, two words	647	5%	0.69	मक्का (<i>makkā</i>) / मकई (<i>makāi</i>)	May be folded to one concept
Unrelated	635	5%		बोवाई (<i>bovāi</i>) / मोबाइल (<i>mobāil</i>)	Dropped from the index

“Conf.” is the mean confidence the judge model attached to that label (Claude Sonnet 5). The same-meaning class has the lowest confidence, which is why step 7 of Appendix A checks that class again before folding anything.

5.4.2 Correction Rule

The rule has one job: repair a farming word the ASR model mangled, and leave every other word alone. It is built to be careful rather than thorough. The reference keeps whatever the farmer actually said and is never tidied up (§6.2), so changing a word that was already right can only lose points. Earlier work on fixing names in transcripts uses spelling distance, sound similarity and sentence context together [24, 25]. This stage uses the first two and leaves anything that needs context alone. Figure 6 is the path a word takes.⁶

- **Which words are looked at.** Only a word real farmers almost never say. If the word appears in the query

⁶The open-source Handy dictation tool [27] was the reference for putting a sound-matching step here. Handy uses Soundex, which does not work on Devanagari, so the sound layer here was built for Indic scripts instead.

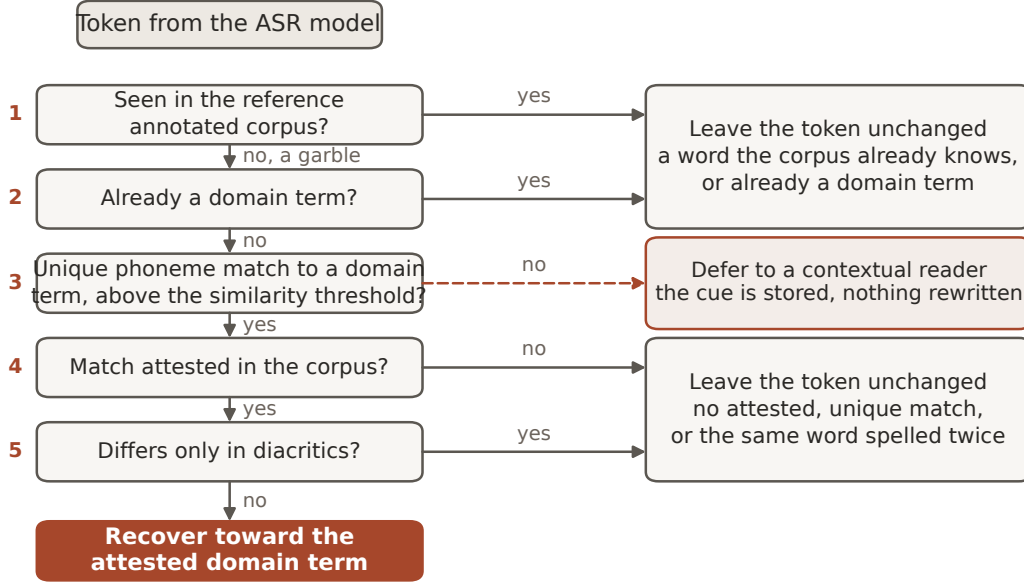


Figure 6: The repair decision path. Structure only; measured effects are in §7.5.

corpus, people use it, so the stage leaves it alone. Words the ASR model effectively made up are the only candidates.

- **How two words are compared.** By the sounds they are spoken with, not the letters they are written with, using a rule that also drops the silent vowel at the end of a Devanagari word. Swapping a soft sound for a hard one is a small difference; moving where in the mouth a sound is made is a bigger one [28, 29].
- **What a word may be changed into.** Only a farming term the ASR model is already known to produce, and only when that one term is a clear winner. If two terms are about equally close, nothing changes.
- **Two spellings of one word.** Devanagari writes some words two ways, a dot where the other spelling puts a small curve. Those are the same word, not a mistake, so they are skipped. Without this the stage would keep rewriting good spelling and dialect variants.
- **Sound-alike words that mean different things are left alone.** The expensive cases, दवा (*davā*) against दावा (*dāvā*) and बीज (*bīj*) against बीच (*bīch*), cannot be settled without reading the sentence, because the wrong word is a real word too. Each is flagged with the list’s cue and passed on unchanged (§9).
- **Why ties are refused.** A quarter of the list sounds exactly like something else in it (Appendix A). Guessing between two farming words that sound the same is worse than leaving the word alone.
- **One sound system, three languages.** The sounds use one Indic scheme rather than the international phonetic alphabet, so the same matcher works for Telugu and Odia as soon as those word lists exist. Today the list is Devanagari and the gate is on the script of each word, so Telugu and Odia script passes through untouched and Devanagari written on a Telugu or Odia clip is repaired as Hindi (Appendix B). Those two word lists are the next build (§9).
- **Settings.** The frequency cut-offs, the closeness threshold and the sound costs are all configuration, not numbers buried in the code (P3).

5.5 M4: Transcript Quality and Recovery

M4 is specified and built but not measured, so it is presented here as design. No result in this paper depends on it. What it is for is the gap of §3.4: no bad transcript should reach the rest of the pipeline silently.

The stage rests on one property of the deployment. The model that writes the farmer’s answer is told which channel the question arrived on, text, voice or image. A model that knows the question was spoken, and knows which words in the transcript of it are in doubt, can behave differently from one handed plain text. That is what makes a recovery step possible without a second pass over the audio.

- **A gate with two parts.** On top of a general confidence score, M4 flags a transcript when an important farming word is shaky: a crop, a chemical, a dose. The weights behind AWWER (§5.4.1) already say which words those are, so the trigger is reasoned rather than one flat threshold. A wrong function word is survivable. A wrong pesticide is not.
- **What it can look at.** ASR model confidence where the provider gives it, the signal features M0 already computed, how many words M3 flagged as different-meaning pairs, and whether the repaired and unrepaired transcripts agree.

- **Recovery happens when the answer is written.** A flagged transcript goes to a higher-reasoning model at response generation, together with the channel and the list of farming words in doubt. That model can hedge, ask for the missing detail, or lean on the rest of the query instead of answering a word it should not trust. Recovery is therefore a property of response generation driven by the channel, not a second pass over the audio. A flagged clip can also be sent to another ASR provider or through a second repair pass.
- **This is where the judgements go that rules cannot make.** Every stage up to here is deterministic on purpose, and that is also its ceiling. A fixed rule cannot tell whether दवा (*davā*) or दावा (*dāvā*) was meant, whether a dose is plausible for the crop named in the same question, or whether an odd word is a garble or a local name. A language model makes those calls well and code does not make them at all. M4 is the one place in the design where such a call belongs, which is what keeps every measured stage free of a model call of its own (§4.3).
- **Thresholds and routing are configuration**, not code (P3), so a deployment sets how much it is willing to flag.
- **Asking the farmer, where the channel allows it.** An uncertain high-stakes word can be put back to the farmer to confirm instead of acted on. This sits outside the single-shot evaluation and affects none of the reported numbers.

We propose the stage and do not test it. What it is worth is how many bad clips it rescues against how often it flags a clip that was already fine, and that needs its own experiment (§9). The closest thing this paper does measure is the weighted domain error of §7.2, because it counts exactly the words M4 is built to catch. Weighted domain error is therefore the proxy for what this module would add: it says how many shaky farming words are still in the transcript for M4 to act on, and it is the metric the M4 experiment should be read against.

6 Experiment Design

6.1 Evaluation Data

Three datasets carry the work. Table 6 lists them once, and we use the names after that. Samples are spread across language, geography, crop, question type, noise level, number of speakers, recording quality and how many farming words a clip holds.

Table 6: The datasets used here. Scored counts per experiment are in each result table’s caption.

Set	Scale	Languages	Role
Published benchmark [7]	10,934 recordings evaluated	Hindi, Telugu, Odia	The earlier evidence this pipeline was built from; released as a dataset [30]
Reference annotated corpus	about 100,000 queries	Hindi	Mines the word list and gives corpus frequency for the repair stage (§5.4)
Human quality-checked corpus (IISc)	about 2,700 clips	Hindi, Telugu, Odia	The evaluation set: the diarizer, selection and the stage ladder; released with both references per clip [2]

A human quality-checked clip joins the evaluation set if it has audio and a farmer reference from the human annotation. The ladder scores the clips where all four stages ran on all four ASR models. Both counts are in the captions of Tables 11 and 13.

6.2 Annotation Protocol

These are the guidelines that answer §3.6, run as a trial with the Indian Institute of Science, Bangalore. Each clip gets six fields:

- **Audio issues**, ticked from a fixed list: background noise, multiple speakers, overlapping speech, cross talk, radio or television, crowd noise, distorted audio, very low volume. More than one tick is normal.
- **Speaker count**, as a number.
- **Transcription with timestamps**, every speaker, as (start -- end) S#: text. A pause over about two seconds or a change of subject starts a new segment. Where two people overlap, whoever is clearly audible is written down.
- **Main speaker**, the label of the person asking the farming question.
- **Farming words** used in the clip.
- **Category** for each of those words: crop, fertilizer, pesticide, disease, soil, irrigation, farming practice or weather.

Table 7 is one real clip with all six fields filled in. S1 is the helper holding the phone, prompting the farmer. S2 is the farmer.

Table 7: One clip as annotated, all six fields. Hindi, two speakers.

Field	Value as annotated
Audio issues	Background noise, multiple speakers
Speakers	2
Transcription	(00:00.8 – 00:02.3) S1: भईया कौन तू क्वेशन पूछब? (<i>brother, which question are you going to ask</i>) (00:02.3 – 00:04.3) S2: सरसो में कीड़ा लगता है, उसका बताइए? (<i>mustard is getting pests, tell me about that</i>) (00:04.8 – 00:05.9) S1: unintelligible
Main speaker	S2
Farming words	सरसों (<i>sarson</i>): crop; कीड़ा (<i>kīṛā</i>): pest
Reference for scoring	S2’s turn only

Because people placed the timestamps, diarization error is measurable on this set directly. Because the main speaker is labelled, so is speaker-selection accuracy.

Only the main speaker’s turns become the reference, because the service has to answer the farmer’s question and not a bystander’s. What was said is kept as said and never tidied up, so editing a word the ASR model already got right can only lose points. That is the condition the repair stage is built around (§5.4.2).

6.3 Text Normalization

Word error rate only means something if the reference and the transcript are cleaned up the same way. In Indic scripts that takes care. The same word can differ by a dot under a letter, by a dot above instead of a small curve, by a number written as digits or as words, or by annotation markup in brackets. So scoring uses the benchmark’s Hindi normalizer, ported unchanged, and runs the same steps for Telugu and Odia rather than adding one-off rules per language.

- **Hindi, ported from the benchmark [7].** Unicode NFC, fold the nukta so both ways of writing a letter collapse to one (क़ to क), turn Devanagari digits into number words the way people say them in the Indian system of crore, lakh, thousand and hundred, strip bracketed annotation and punctuation, collapse whitespace.
- **Telugu and Odia, added here.** The same steps, each with its own digit map and number-word table, checked against AI4Bharat’s `indic-numtowords` convention. Odia folds its own nukta, and both add the Indic danda to the punctuation list.
- **Corpus-level scoring.** WER is pooled: total edits over total reference words, as in the benchmark, rather than an average of per-clip rates. Each clip is normalized by its own language.
- **Shared sound space.** The repair stage (§5.4.2) uses the same normalization plus schwa deletion by rule, which is what lets one word list and one matcher cover all three scripts as the other lists grow.

The Telugu and Odia number-word tables are built by rule and not yet reviewed by a native speaker, so absolute WER in those two languages is comparable inside this paper rather than against outside figures. Stage-to-stage and model-to-model differences are unaffected, because the same normalizer runs on both sides.

6.4 Pipeline Configurations

The main protocol is a ladder. Each stage keeps everything before it and adds one thing (Table 9). Table 8 names the modules those columns stand for.

1. **Everything else is held still.** Same clips, same ASR model, same reference at every step. The only thing that changes is the audio or the text handed to the ASR model.
2. **Selection sits with diarization in S3,** because finding the turns and picking the farmer are one operation.
3. **Repair stands alone in S4,** so its effect is read on its own.
4. **Selection and slicing read the audio the cleaning gate passes on,** so the ladder measures the stages in the order a deployment runs them.
5. **M4 is measured on its own terms.** What it is worth is how many bad clips it rescues, not accuracy on the clips it accepts, so it needs its own experiment (§9) rather than a rung on this ladder.

Table 8: The five modules the ladder columns stand for.

Module	What it does
M0	Audio analysis and gated cleaning (§5.1)
M1	Speaker detection and selection (§5.2)
M2	ASR model, unchanged and replaceable (§5.3)
M3	Domain-aware correction against the farming word list (§5.4)
M4	Transcript quality and recovery, specified in §5.5

Table 9: The four ladder stages and what each hands the ASR model. ✓ on, ✗ off.

Stage	Input to the ASR model	M0	M1	M2	M3
S1 Baseline	Raw audio, no cleaning	✗	✗	✓	✗
S2 Enhancement	Audio after gated cleaning (DFN3, 6 dB)	✓	✗	✓	✗
S3 Diarization and selection	The farmer’s turns only, found by the fine-tuned segmenter and speaker selection	✓	✓	✓	✗
S4 Domain correction	The farmer’s turns, then repair against the farming word list (§5.4.2)	✓	✓	✓	✓

The ladder is cumulative: each stage keeps the ones before it, so a stage’s contribution is the step it adds. S1 is Baseline 0 of §2.2. What a module adds on its own comes from the study built for it: cleaning from the S1 to S2 comparison on the clips where it fired, sliced by noise tier (§7.3); selection from the diarizer comparison and the selector pilot (§7.4); repair from the per-edit audit (§7.5).

6.5 Metrics

We keep two kinds of error apart all the way through. Did the transcript match the reference, and did the farmer’s question survive. One example shows why both are needed: when खाद (*khād*), fertilizer, comes back as खाद्य (*khādyā*), food, WER charges one word, the same price as a dropped function word, while the farming-term rate charges the word the whole question was about (Table 1). So a weighted rate and a farming-term rate are always reported next to WER, and never one of them alone (Table 10).

Table 10: What each metric measures and where it runs.

Metric	What it measures	Where it runs
Corpus WER	Wrong words against the farmer reference, pooled over the corpus	Every rung, and M0, M2, M3
Farmer-query F1	Recall against precision on the farmer reference, per clip	Every rung, and M0, M2, M3
AWWER	WER with every error charged the importance of its word (§6.6)	M3 and the ladder
Farming-term rates	Whether the crop, pest or chemical word survived, by word-list lookup	M3 and the ladder
DER	Missed speech, false alarm and speaker confusion, at a 0.25 s collar	M1
cpWER	Error after each speaker is matched to a reference speaker [23]	M1
Main-speaker accuracy	How often selection picks the labelled main speaker	M1
Latency and cost	What each stage adds per query, and what it takes to deploy	Every stage
Answer quality	Out of scope: it rests on the model that reads the transcript	Not measured

WER reads a removed bystander the same as a removed farmer, and farmer-query F1 is what separates them (Appendix B). Farming terms are extracted with spelling variants folded on both sides, so a variant spelling counts as a match instead of looking like a rescue.

Two markers appear throughout. A value measured on a sample rather than the full evaluation set carries ^P, with its sample size where it is quoted. A value a language-model classifier produced, rather than code, carries ^L and keeps that marker until people confirm it.

6.6 Error-Cost-Aware Evaluation

Word error rate counts the insertions, deletions and substitutions needed to turn the transcript into the reference, divided by the number of reference words. Every word costs the same, so a lost crop name and a lost function word are charged alike. AWWER weights the errors by how much they matter instead, using the four tiers of §5.4.1:

$$\text{AWWER} = \frac{\sum_{i \in \text{errors}} w_i}{\sum_{j \in \text{reference}} w_j}, \quad (1)$$

where w is the weight from word-list lookup and anything not in the list gets weight 1 [7, §3.3.2].

- **Fixed and mechanical.** Words are lined up against the reference and charged by weight, with no language-model call at scoring time, though the word list the weights come from was adjudicated by one^L.
- **A reference word replaced or dropped** is charged its own importance.
- **A word the ASR model added** is charged its own importance too, so an invented crop name costs and an invented function word does not.
- **Weights are assigned once.** A judge model weights each word list entry, code applies it after that, and the judge is never the same model as the ASR model being tested.

AWWER does not by itself close the gap in §3.5. An ASR model can look good on AWWER and still fail the meaning check, which is why the farming-term rate of §6.5 is reported with it.

6.7 Run Coverage

The ladder ran to completion: the full evaluation set, all four stages, all four ASR models, all three languages.

- **ASR models.** Four off-the-shelf systems from different families and deployment styles, run through the identical pipeline: Gemini 3.7 Flash [31] and Sarvam saaras:v3 [32] (cloud), Azure Speech [33] (cloud), and IndicConformer 600M [34] (on-device CTC). The model-agnostic claim is measured on all four.
- **Languages.** All three tested together on the full set. Repair is Hindi first, because the word list and its sound rules are Devanagari. It is gated on the script of each word rather than the language of the clip, so it also edits the Devanagari an ASR model sometimes produces on a Telugu or Odia clip (Appendix B).
- **What each ASR model brings.** Gemini 3.7 Flash is cloud and generative and shows the biggest multi-speaker gain, Sarvam saaras:v3 is cloud and Indic-specialized, Azure Speech runs with cleaning on in the ladder even though its own evidence argues for switching it off, and IndicConformer 600M is on-device CTC with no API cost. No cell of the ladder is missing.

7 Results

The ladder of §6.4 ran to completion on the full evaluation set and on all four ASR models. Every value here is a measured corpus value, produced from the committed per-clip result files. Values from a sample carry ^P and values a judge model produced carry ^L.

7.1 The Stage Ladder, S1 to S4

The pipeline beats the plain ASR model on all four we tested, and the difference is statistically significant each time. Against raw audio, the full pipeline cuts corpus WER by 16% to 23% on the three cloud ASR models and by 5% on the on-device model. On clips with more than one speaker it cuts error by 16% to 42%. One stage carries almost all of that, and the on-device model gains the least (Figure 7, Table 11).

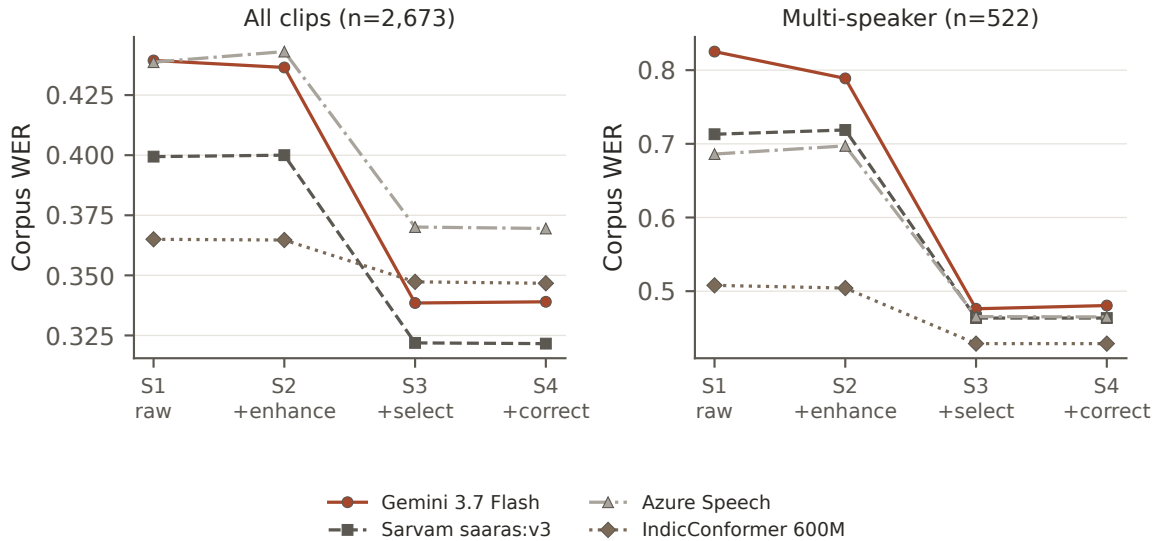


Figure 7: Corpus WER across the four stages, one line per ASR model, all clips and multi-speaker clips.

Table 11: The stage ladder: corpus WER and farmer-query F1. Best value per row in green bold.

Subset	ASR model	S1	S2	S3	S4
		WER / F1	WER / F1	WER / F1	WER / F1
All ($n = 2,673$)	Gemini 3.7 Flash	0.439 / 0.696	0.436 / 0.695	0.339 / 0.700	0.339 / 0.700
	Sarvam saaras:v3	0.399 / 0.720	0.400 / 0.720	0.322 / 0.729	0.322 / 0.729
	Azure Speech	0.439 / 0.665	0.443 / 0.664	0.370 / 0.682	0.369 / 0.683
	IndicConformer 600M	0.365 / 0.678	0.365 / 0.679	0.347 / 0.683	0.347 / 0.684
Multi-speaker ($n = 522$)	Gemini 3.7 Flash	0.825 / 0.617	0.789 / 0.612	0.476 / 0.656	0.480 / 0.656
	Sarvam saaras:v3	0.713 / 0.625	0.719 / 0.625	0.464 / 0.662	0.464 / 0.662
	Azure Speech	0.686 / 0.591	0.697 / 0.590	0.466 / 0.642	0.465 / 0.643
	IndicConformer 600M	0.508 / 0.619	0.504 / 0.624	0.429 / 0.641	0.429 / 0.641
Single-speaker ($n = 2,151$)	Gemini 3.7 Flash	0.337 / 0.715	0.343 / 0.715	0.302 / 0.710	0.301 / 0.711
	Sarvam saaras:v3	0.316 / 0.743	0.315 / 0.743	0.284 / 0.745	0.284 / 0.745
	Azure Speech	0.373 / 0.683	0.376 / 0.682	0.345 / 0.692	0.344 / 0.692
	IndicConformer 600M	0.327 / 0.692	0.328 / 0.693	0.326 / 0.693	0.325 / 0.694

Three things hold for all four ASR models.

- **Speaker selection carries the pipeline.** It is the only stage with a big, reliable drop in error, and the drop is biggest where several people speak (Table 12). On multi-speaker clips it removes between a sixth and nearly half of the word error. Farmer-query F1 goes up at the same time, so the gain is not just words being deleted.
- **Cleaning pays on the noisy tail.** At corpus level its effect sits inside the interval on three of the four ASR models, and on Azure it is a small reliable increase (Table 12). The gain is in the high-noise clips and depends on the model (§7.3), which is what the noise gate and the per-model switch are for.
- **Repair is conservative, and its value is in farming terms.** S3 to S4 is a small reliable drop on Azure and IndicConformer and a wash on Gemini and Sarvam, so it runs without a gate; on Gemini’s multi-speaker clips it is a small reliable increase. What it is for is recovering garbled farming words, which it does on all four (§7.5, Appendix B).

Table 12: Change in corpus WER per stage, paired clip bootstrap.
Negative is better; * marks significance. Every S1 to S4 change is significant.

<i>All clips</i>				<i>Multi-speaker clips</i>			
ASR model	S1→S2	S2→S3	S3→S4	ASR model	S1→S2	S2→S3	S3→S4
Gemini 3.7 Flash	−0.003	−0.098*	+0.001	Gemini 3.7 Flash	−0.036	−0.313*	+0.004*
Sarvam saaras:v3	+0.001	−0.078*	0.000	Sarvam saaras:v3	+0.006	−0.255*	0.000
Azure Speech	+0.004*	−0.073*	−0.001*	Azure Speech	+0.011	−0.232*	−0.001
IndicConformer 600M	0.000	−0.017*	−0.001*	IndicConformer 600M	−0.004	−0.075*	0.000

7.2 AWWER

WER charges the same for a lost crop name as for a lost function word: a dropped दवा (*davā*), the pesticide, costs exactly what a dropped postposition costs. AWWER exists to fix that (§6.6). Under AWWER the ladder tells the same story as under WER: selection lowers weighted error on every ASR model, and repair leaves it where selection put it (Table 13). The weights come from a Devanagari word list, so they cover 88.6% of Hindi reference words and charge Telugu and Odia words flat until those lists are built (§9). The Hindi column is the weighted view. AWWER is below WER everywhere in that table. That says something about the errors, not about the metric: it means the remaining errors sit on words that matter less than the average reference word, so more of them are function words than crop, pest or chemical names. Because an invented word is now charged its own importance instead of a flat 1, AWWER can be compared across stages and across ASR models that differ in how much they write down.

Table 13: AWWER / WER pairs across the ladder, lexicon weights covering Hindi only. Hindi $n = 1,130$, all languages $n = 2,677$.

ASR model	Hindi, AWWER / WER			All languages, AWWER / WER		
	S1	S3	S4	S1	S3	S4
Gemini 3.7 Flash	0.319 / 0.395	0.224 / 0.266	0.224 / 0.266	0.361 / 0.439	0.272 / 0.339	0.273 / 0.339
Sarvam saaras:v3	0.330 / 0.384	0.255 / 0.283	0.255 / 0.282	0.351 / 0.399	0.282 / 0.322	0.282 / 0.322
Azure Speech	0.349 / 0.416	0.272 / 0.308	0.271 / 0.307	0.377 / 0.439	0.312 / 0.370	0.312 / 0.369
IndicConformer 600M	0.278 / 0.317	0.259 / 0.287	0.259 / 0.286	0.312 / 0.365	0.297 / 0.347	0.296 / 0.347

AWWER weights the errors but still scores the whole transcript. The narrower question, whether the farmer’s own crop, pest or chemical word survived, is measured directly in Appendix B: selection trades a little farming-term recall for more precision, and repair raises that recall on every ASR model.

7.3 S2 Enhancement

Cleaning depends on the ASR model, and that dependence is the finding. On the high-noise clips it lowers pooled WER for the generative cloud ASR model by 0.147, does almost nothing for the on-device model at −0.010, and makes things worse for Sarvam (+0.061) and Azure (+0.034) (Figure 8). Two readings are both true, and they answer different questions. Pooled over the noisy clips, Gemini’s gain is real and big enough to switch the stage on for that ASR model. Clip by clip, the median change is zero in every noise band for every ASR model, and about as many clips get worse as get better (Table 14), so the pooled gain comes from a minority of clips rather than a general shift.

“Pooled” is the change in corpus WER. The middle per-clip change is zero in every cell, which is the point of the last column: cleaning leaves most clips alone and moves a few a long way.

The default setting costs error on the tier it should help, so 6 dB is the recommended cap^P (200 clips).

So the gate is doing its job. Cleaning fires only above the noise threshold and only for an ASR model whose own evidence supports it. It helps the model it helps and leaves the other two within a thousandth of where they started at corpus level. Azure, where cleaning costs a reliable 0.004, is the case the gate exists to switch off.

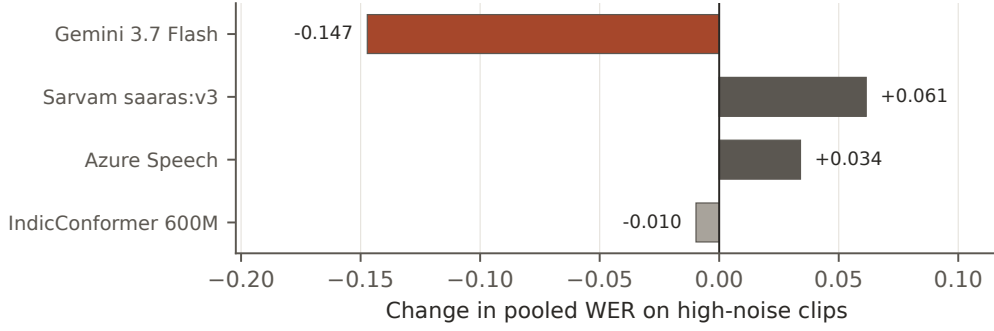


Figure 8: Change in pooled WER from gated cleaning on high-noise clips, per ASR model. Negative is better.

Table 14: Gated cleaning on the full evaluation set, per ASR model. Negative is better.

ASR model	High noise			Medium noise		
	Pooled	Better / worse / same		Pooled	Better / worse / same	
Gemini 3.7 Flash	-0.147	28 / 20 / 46		+0.008	158 / 164 / 231	
Sarvam saaras:v3	+0.061	15 / 12 / 67		-0.007	64 / 45 / 444	
Azure Speech	+0.034	20 / 18 / 56		+0.016	93 / 110 / 350	
IndicConformer 600M	-0.010	15 / 11 / 68		+0.000	77 / 73 / 403	

7.4 S3 Diarization and Selection

The part that makes S3 work is the diarizer. Nine systems were scored, seven open and two cloud, and none of them split this audio cleanly, which is why one component was fine-tuned. In the scored ladder selection runs on every clip, and the gate a deployment uses to skip the single-speaker majority is measured on its own below. The diarizer values here are on a frozen held-out split, every system against the same reference at the same 0.25 s collar.

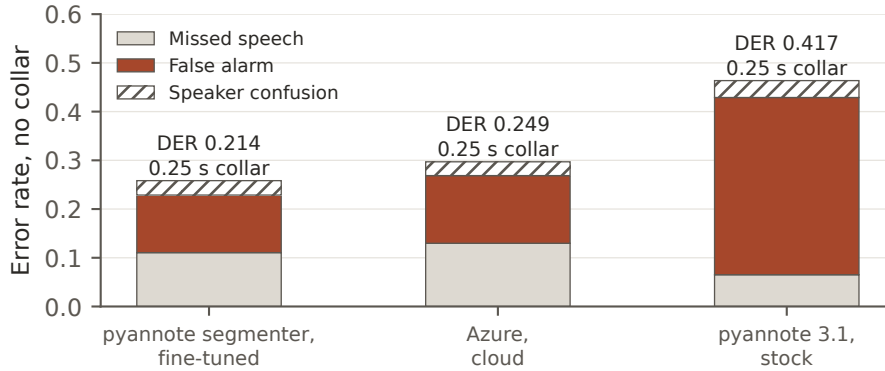


Figure 9: Diarization error by component on the held-out split. Bars carry no collar, the annotation does.

- **The fine-tuned segmenter is the most accurate.** It reaches DER 0.214, against 0.249 for the best cloud diarizer and 0.417 for stock pyannote (Figure 9).
- **What fell is false alarm**, which is speech heard in background noise. It drops from 0.364 of stock pyannote’s error to 0.118, paid for with missed speech rising from 0.065 to 0.111. Components are scored without a collar, so they sum above the collared totals. The trade suits the common case of one bystander, and §8.1 says what a deployment with more group calls changes.
- **A lower DER does not mean a better transcript.** With the ASR model held fixed on each diarizer’s segments, DER and cpWER move together only loosely (Pearson $r = 0.60$, Spearman $\rho = 0.57$). NeMo MSDD is mid-pack on DER and second-best on cpWER, ECAPA is mid-pack on DER and worst on cpWER, and the fine-tuned segmenter’s cpWER of 0.435 is a near tie with Azure’s 0.431 despite a clear DER lead (Figure 10).
- **So diarizers are ranked by cpWER** with a fixed ASR model. Hearing too much speech costs DER seconds but barely costs words, which is why the segmenter was fine-tuned instead of taking whichever model had the lowest DER.
- **What selection removes** is the person in Table 7 who holds the phone and prompts the farmer. Text-only rules find the labelled main speaker well across the corpus and clearly worse on the multi-speaker clips that matter (Table 15).
- **Ungated, selection costs the farmer’s own words.** On single-speaker clips there is no bystander to

remove, and slicing to turns cuts the farmer at the edges: a farmer who starts talking before the segmenter marks the turn loses those opening words. On 90 single-speaker pilot clips WER falls from 0.455 to 0.374 while farmer-query F1 falls from 0.712 to 0.667^P.

- **The gate that ships needs no labels.** Run selection only when a stock diarizer sees a second speaker hold the floor for at least a second. On 149 pilot clips that gate agrees with the annotated speaker count 76% of the time, lifts all-clip F1 from 0.678 to 0.686 and lowers WER from 0.630 to 0.422^P. On a bare speaker count, with no duration floor, it is 69% accurate and lands below doing nothing at all.

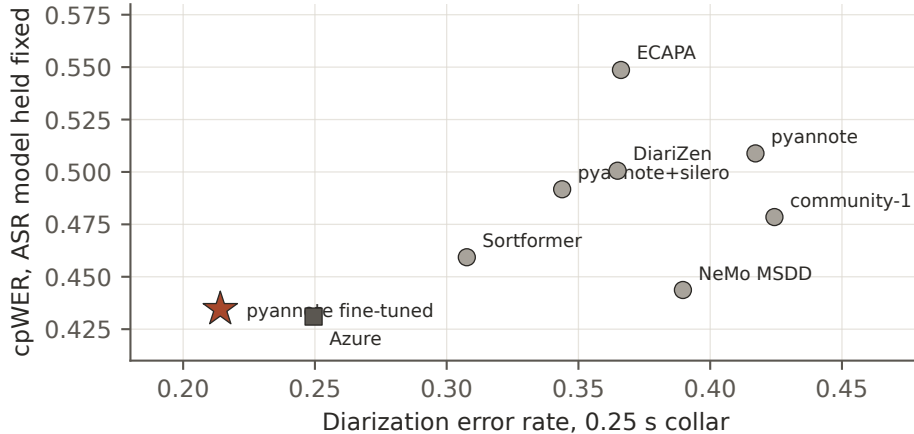


Figure 10: Diarization error against cpWER with the ASR model held fixed, on the held-out split. Bottom left is better.

Table 15: Text-only speaker-selection rules against the annotated main-speaker label, $n = 2,716$ clips. Best per column in green bold.

Rule	All clips	Multi-speaker clips
Longest single turn	0.949	0.770
Most total speech	0.944	0.747
Most turns	0.938	0.713
Earliest speaker	0.882	0.428
Random choice	0.879	0.413

7.5 S4 Domain Correction

The repair stage is deliberately narrow, and Table 16 measures how narrow.

- **It touches few clips.** Across all four ASR models it changes 194 clips: 37 get better on corpus WER, 17 get worse, and 140 do not move.
- **The kind of edit it makes.** On a wheat query, खरपतवा (*kharpatvā*), which is not a word, goes back to खरपतवार (*kharpatvār*), weed, which is what the farmer’s own reference says. A word the corpus does use is left alone, whatever it sounds like.
- **The harm is small, and worth reporting.** Those 17 are 0.16% of all clip-ASR model pairs and 8.8% of the clips the stage touched. Touching only words the corpus has never seen is what keeps that rate this low without a gate, and wrongly rewriting a crop or chemical name is worse than not repairing at all.
- **The harm is not spread evenly.** 14 of Gemini’s 41 edits raise WER, against none of Azure’s 41. Gemini writes down more of what it hears, so a word the corpus has never seen is more often a real rare word than a garble.
- **It does most on the on-device model.** IndicConformer garbles more farming words than the cloud systems do, so one word list rescues more of them: the stage touches 70 of its clips against 41 or 42 for the others, and raises its farming-term recall (Appendix B). That is the ASR model a deployment cannot simply swap for a stronger cloud API.

Table 16: What the repair stage touches, per ASR model, on the 2,677 scored clips.

ASR model	Clips changed	WER improved	WER worsened	WER unchanged
Gemini 3.7 Flash	41	6	14	21
Sarvam saaras:v3	42	8	2	32
Azure Speech	41	11	0	30
IndicConformer 600M	70	12	1	57
All four	194	37	17	140

What the stage has to leave alone is not spread evenly across the word list. Grouped by category, the different-meaning pairs of §5.4.1 pile up in the categories the weighting scores highest, farming practice and crops above all, because a word farmers ask about often is a word the ASR model mishears often (Figure 11). Those are the pairs repair flags and never rewrites, so the categories that matter most are also the ones where a contextual pass would buy the most (§9).

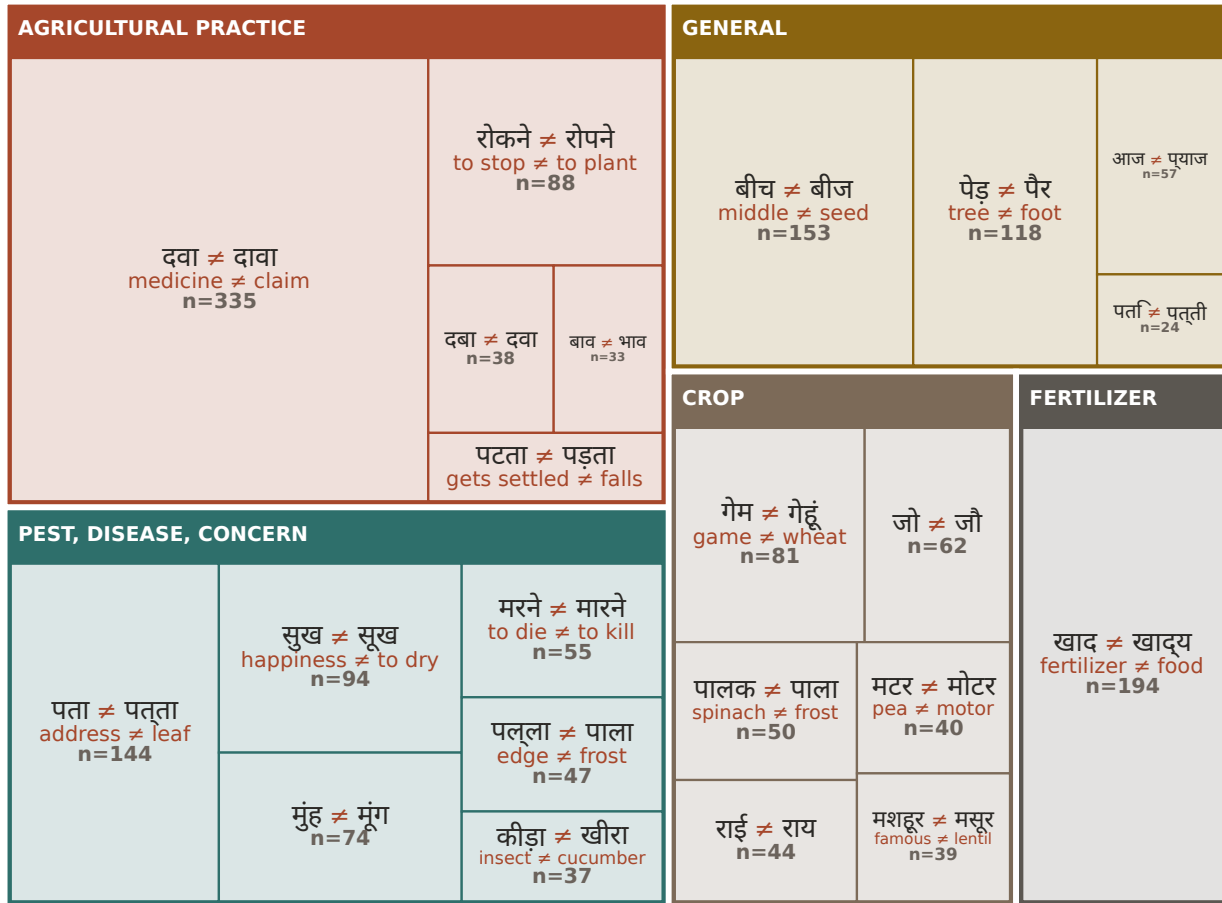


Figure 11: Different-meaning confusions by category. Tile area is the observed count.

7.6 Cost and Latency

The pipeline adds a small, mostly CPU-bound cost around an unchanged ASR model (Table 17). No stage adds a second ASR call, and repair adds no model call at all.

Table 17: What each stage adds in latency and cost. The ASR model's own bill does not change.

Stage	Latency	Cost
M0 cleaning	Real-time factor about 0.02, roughly 113 ms median on a clip where it fires. The gate skips about 76% of traffic as already clean, so about 27 ms per query on average	Cleaner is CPU only; the deployment gate is waveform arithmetic, and the runs here read the corpus noise tier
M1 diarization and selection	Real-time factor 0.019, estimated for the pyannote family on one A10G GPU	About \$0.02 per audio-hour self-hosted, that factor at full GPU use counting compute only, against \$0.18 per audio-hour published for the cloud service
M2 ASR model	Unchanged	Unchanged; no API cost at all with the on-device model
M3 repair	Text only, no extra ASR or model call	No model call

8 Discussion

8.1 Remaining Failure Modes

Four failures are left, each with a route out named against it. The places where a stage moves error the wrong way are reported with that stage in §7, and each is why the stage is gated the way it is.

- **Sound-alike words on clean audio.** When the wrong word is also a real word, only the sentence can tell you which was meant. The fixed rules leave these alone on purpose, for a reader that can use context (Table 5, §9).
- **Clips with three or more speakers.** The fine-tuned segmenter accepts a little more missed speech in exchange for a large drop in false alarms, which is the right trade for the one bystander that most clips carry (§7.4). The threshold is configuration (P3), so a deployment that sees more group calls can move it back toward recall.
- **Words lost at turn edges.** Cutting the audio down to the farmer’s turns costs a little of the farmer’s own farming vocabulary (Appendix B). The selection errors that remain are cases where a bystander simply talks longer than the farmer. The next lever is a content cue, since the question usually names the crop or the problem.
- **A dropped word cannot be repaired.** If the ASR model never wrote the farming word down, there is nothing to fix. Only ASR-side routes help there: cleaning, a vocabulary hint at decode time, or asking the ASR model for several candidates.

Odia is the hardest language for the generative ASR model. It has the most high-noise audio (Figure 2), the largest raw error on the farmer reference, and it is still the hardest of the three after all four stages (Table 20).

A better transcript is not automatically a better answer. In a small pilot, enhanced and unenhanced transcripts went through the whole advisory path and were compared by a blind judge with position controlled; the transcript gain on high-noise clips did not change which answer was preferred, because on the hardest clips both transcripts are wrong in different ways^{PL}. None of these failures is silent by design, which is why the quality gate (M4) sits in the architecture (§5.5); measuring what it rescues is the next experiment (§9).

8.2 Generalizability

- **Works across ASR models.** The same pipeline, unchanged, improves all four ASR models significantly, and the gain is biggest where the audio is hardest and where the ASR model writes down the most bystander speech, without hurting the easy single-speaker case.
- **Works across languages.** Cleaning, diarization and selection listen to audio, so they behave the same in all three languages. Repair reaches Devanagari today, and because it is built on a sound alphabet shared across scripts the same matcher extends as the Telugu and Odia word lists are built (§9).
- **Across geographies.** The next test is field audio at benchmark scale from another region (§9).
- **Beyond farming.** The shape of the system does not depend on the domain: wrap an untouched ASR model, aim one specialist stage at each failure, and switch each stage on only where it helps. Only the word list is domain-specific, and the way it is built works for any other controlled vocabulary (Appendix A).

8.3 Engineering Implications

- Every stage is a separate part behind a shared interface, so a team can roll this out piece by piece. Add speaker selection first, because it is the biggest lever. Leave cleaning off for a ASR model it hurts. Swap the model without touching anything else.
- The extra cost is small and mostly CPU work (Table 17), which matters for a live service where an accuracy gain that multiplies latency or the bill cannot ship.
- Provider independence is the practical payoff. The ASR model is the one swappable cost centre, and the stages around it carry the domain knowledge that would otherwise force a fine-tune, so a deployment can trade accuracy against cost without a redesign.

9 Future Work

Each item below is scoped, and most are already specified. This is what the pipeline needs next.

- **Measure M4.** How many bad clips the quality gate rescues, against how often it rescues a clip that was already fine.
- **A contextual pass for sound-alike words.** Let the model that reads the transcript settle the pairs the fixed rules only flag (Table 5).
- **Repair in Telugu and Odia.** Each needs a mined word list, labelled pairs and its own sound layer before S4 means anything.

- **More languages.** Amharic, Afaan Oromo, Swahili and Hausa next, each with its own word list and sound layer.

10 Conclusion

General-purpose ASR models transcribe noisy, multi-speaker, language-mixed farmer audio poorly, and the words they lose are the crop, pest and chemical names that carry the question. Five cheap stages, one of them the untouched ASR model, fix much of that: clean the audio when it needs it, split the speakers and keep the farmer, recognize with the ASR model unchanged, repair garbled farming words against a checked word list, and judge the transcript before it goes on. No ASR model was retrained, no provider was replaced, and no large agent system was built, and no stage of the pipeline makes a language-model call of its own.

On human quality-checked field recordings in Hindi, Telugu and Odia it cuts word error rate by 16% to 23% on the three cloud ASR models and by 5% on the on-device model, and by 16% to 42% on multi-speaker audio. Every one of the four ASR models improves significantly. That the same pipeline works across model families and across cloud and on-device deployment is what makes the claim about ASR model independence real rather than asserted. The big lever is isolating the farmer’s own speech. Cleaning and repair are switched on only where the evidence supports them, and that evidence came from measurement, not assumption. One component is fine-tuned, the diarization segmenter, because nothing off the shelf worked across these three languages. Every other stage is off the shelf behind a shared interface.

Two findings carry beyond this system. Cleaning up audio does not reliably help recognition, so a stage that makes audio sound better has to be gated on what the ASR model does with it. And when the reference keeps the speaker’s own words, repair has to aim at precision rather than coverage. The pipeline is released with its word list and its evaluation code, and its stages can be deployed and swapped one at a time.

11 Open-Source Release

Four artifacts are public now: the evaluation set behind every result in §7, the farming word list, the one fine-tuned checkpoint, and a demonstration of the pipeline on sample clips. The pipeline and evaluation code is published with this paper. Table 18 lists each one with its licence.

Table 18: The open-source release, with licence and availability per artifact.

Artifact	What it holds	Licence	Available
Evaluation set [2]	Field audio with human transcripts in Hindi, Telugu and Odia. Every clip carries both the farmer reference used for scoring here and the full per-speaker transcription with turn timestamps, plus speaker count and audio issues	CC-BY-4.0	Now
Farming word list [3]	The weighted terms, the adjudicated sound-alike pairs with their cues, the candidate neighbours and the variant families of Appendix A	CC-BY-4.0	Now
Diarization segmenter [4]	The fine-tuned checkpoint, which runs on CPU, with its training recipe and its held-out result	MIT	Now
Demonstration [6]	The pipeline stage by stage on sample clips: raw, cleaned and farmer-only audio, with each transcript against the human reference	n/a	Now
Pipeline and evaluation code [5]	The stage implementations, the ladder runner, the diarizer comparison, the corrector, the significance testing, and the figure and table generators	n/a	With this paper

With the code, the evaluation set, the word list and the checkpoint, every value in the result tables of §7 regenerates from the committed per-clip result files. The field audio the segmenter trained on is not redistributed, so that artifact ships as weights and recipe rather than as a training set. The earlier benchmark this work builds on is published separately [30].

References

- [1] Sanyam Singh, Naga Ganesh, Vineet Singh, Lakshmi Pedapudi, Ritesh Kumar, S. S. P. Jyothi, Archana Karanam, Waseem Pasha, Ekta Kumari, C. Yashoda, Mettu Vijaya Rekha Reddy, Shesha Phani Debbesa, and Chandan Dash. Fine-tuning and evaluating Conversational AI for agricultural advisory. *arXiv preprint arXiv:2603.03294*, 2026.
- [2] Digital Green. Agricultural voice evaluation set (Hindi, Telugu, Odia). Hugging Face Datasets, 2026. CC-BY-4.0. <https://huggingface.co/datasets/DigiGreen/agri-voice-eval>.
- [3] Digital Green. Agricultural domain lexicon (Hindi). Hugging Face Datasets, 2026. CC-BY-4.0. <https://huggingface.co/datasets/DigiGreen/agri-lexicon-hindi>.

- [4] Digital Green. pyannote segmentation fine-tuned for Indic agricultural field audio. Hugging Face Models, 2026. MIT. <https://huggingface.co/DigiGreen/pyannote-segmentation-agri-indic>.
- [5] Digital Green. Agricultural voice pipeline: Stages, ladder runner and evaluation harness. Source code repository, 2026. Published with this paper. <https://github.com/aakashdg/agri-voice-pipeline>.
- [6] Digital Green. FarmerChat voice pipeline demonstration. Hugging Face Spaces, 2026. <https://huggingface.co/spaces/DigiGreen/farmerchat-voice-pipeline-demo>.
- [7] Chandrashekar M S, Vineet Singh, and Lakshmi Pedapudi. Benchmarking Automatic Speech Recognition for Indian Languages in agricultural contexts. *arXiv preprint arXiv:2602.03868*, 2026.
- [8] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [9] Vineel Pratap, Andros Tjandra, Bowen Shi, Paden Tomasello, Arun Babu, Sayani Kundu, Ali Elkahky, Zhaoheng Ni, Apoorv Vyas, Maryam Fazel-Zarandi, Alexei Baevski, Yossi Adi, Xiaohui Zhang, Wei-Ning Hsu, Alexis Conneau, and Michael Auli. Scaling speech technology to 1,000+ languages. *arXiv preprint arXiv:2305.13516*, 2023.
- [10] Ministry of Electronics and Information Technology, Government of India. Bhashini: National language translation mission, 2022. <https://bhashini.gov.in>.
- [11] ART PARK, Indian Institute of Science Bangalore and Google. Vaani: Large-scale multi-modal speech data collection for indian languages, 2023. <https://vaani.iisc.ac.in>.
- [12] Tahir Javed, Sumanth Doddapaneni, Abhigyan Raman, Kaushal Santosh Bhogale, Gowtham Ramesh, Anoop Kunchukuttan, Pratyush Kumar, and Mitesh M. Khapra. Towards building ASR systems for the next billion users. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 10813–10821, 2022.
- [13] Harveen Singh Chadha, Anirudh Gupta, Priyanshi Shah, Neeraj Chhimwal, Ankur Dhuriya, Rishabh Gaur, and Vivek Raghavan. Vakyansh: ASR toolkit for low resource indic languages. *arXiv preprint arXiv:2203.16512*, 2022.
- [14] Nithya R, Malavika S, Jordan F, Arjun Gangwar, Metilda N J, S. Umesh, et al. SPRING-INX: A multilingual indian language speech corpus by SPRING lab, IIT madras. *arXiv preprint arXiv:2310.14654*, 2023.
- [15] Hendrik Schröter, Tobias Rosenkranz, Alberto N. Escalante-B., and Andreas Maier. DeepFilterNet: Perceptually motivated real-time speech enhancement. In *Proceedings of Interspeech*, 2023. arXiv:2305.08227.
- [16] Hervé Bredin. pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe. In *Proceedings of Interspeech*, 2023.
- [17] Alexis Plaquet and Hervé Bredin. Powerset multi-class cross entropy loss for neural speaker diarization. In *Proceedings of Interspeech*, 2023.
- [18] Brecht Desplanques, Jenthe Thienpondt, and Kris Demuynck. ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification. In *Proceedings of Interspeech*, 2020.
- [19] Tae Jin Park, Nithin Rao Koluguri, Jagadeesh Balam, and Boris Ginsburg. Multi-scale speaker diarization with dynamic scale weighting. In *Proceedings of Interspeech*, 2022.
- [20] Tae Jin Park, Ivan Medennikov, Kunal Dhawan, Weiqing Wang, He Huang, Nithin Rao Koluguri, Krishna C. Puvvada, Jagadeesh Balam, and Boris Ginsburg. Sortformer: A novel approach for permutation-resolved speaker supervision in speech-to-text systems. *arXiv preprint arXiv:2409.06656*, 2024.
- [21] Jiangyu Han, Federico Landini, Johan Rohdin, Anna Silnova, Mireia Diez, and Lukáš Burget. Leveraging self-supervised learning for speaker diarization. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025. arXiv:2409.09408.
- [22] Silero Team. Silero VAD: pre-trained enterprise-grade voice activity detector. <https://github.com/snakers4/silero-vad>, 2024.
- [23] Shinji Watanabe, Michael Mandel, Jon Barker, Emmanuel Vincent, Ashish Arora, Xuankai Chang, Sanjeev Khudanpur, Vimal Manohar, Daniel Povey, Desh Raj, David Snyder, Aswin Shanmugam Subramanian, Jan Trmal, Bar Ben Yair, Christoph Boeddeker, Zhaoheng Ni, Yusuke Fujita, Shota Horiguchi, Naoyuki Kanda, Takuya Yoshioaka, and Neville Ryant. CHiME-6 challenge: Tackling multispeaker speech recognition for unsegmented recordings. In *Proceedings of the 6th International Workshop on Speech Processing in Everyday Environments (CHiME 2020)*, 2020. arXiv:2004.09249.
- [24] Abhinav Garg, Ashutosh Gupta, Dhananjaya Gowda, Shatrughan Singh, and Chanwoo Kim. Hierarchical multi-stage word-to-grapheme named entity corrector for automatic speech recognition. In *Interspeech*, pages 1793–1797, 2020.
- [25] Rao Ma, Mengjie Qian, Mark Gales, and Kate Knill. ASR error correction using large language models. *arXiv preprint arXiv:2409.09554*, 2024.
- [26] Andrew C. Morris, Viktoria Maier, and Phil Green. From WER and RIL to MER and WIL: improved evaluation measures for connected speech recognition. In *Proceedings of Interspeech*, 2004.
- [27] C. J. Pais. Handy: An open-source speech-to-text dictation tool. Software, MIT licence, 2025. <https://github.com/cjpais/Handy>.
- [28] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966.

- [29] Grzegorz Kondrak. A new algorithm for the alignment of phonetic sequences. In *Proceedings of the 1st Meeting of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 288–295, 2000.
- [30] Digital Green. Agri STT benchmarking dataset. Hugging Face Datasets, 2025. https://huggingface.co/datasets/DigiGreen/Agri_STT_Benchmarking_Dataset.
- [31] Google. Gemini 3.7 Flash model documentation, 2026. Vendor documentation. Versions used in the component measurements: Gemini 2.5 Pro, 3.5 Flash, 3.6 Flash, 3.7 Flash. <https://ai.google.dev/gemini-api/docs/models>.
- [32] Sarvam AI. Sarvam speech-to-text API documentation, 2026. Vendor documentation. Version used: saaras:v3. <https://docs.sarvam.ai>.
- [33] Microsoft. Azure AI Speech service documentation, 2026. Vendor documentation. <https://learn.microsoft.com/azure/ai-services/speech-service/>.
- [34] AI4Bharat. IndicConformer 600M multilingual. Hugging Face Models, 2025. <https://huggingface.co/ai4bharat/indic-conformer-600m-multilingual>.

A Agricultural Lexicon Construction

The word list of §5.4.1 is built in seven steps from roughly 100,000 real Hindi farmer questions. Every step works from word frequency or from sound, so the same build runs for another language once a comparable corpus exists. All counts are read from a committed file, and the products are released [3]. Figure 4 carries an example through each stage.

1. **Mine.** Pull candidate farming words and phrases out of the query corpus, ranked by frequency. Single words and two-to-three word phrases both count, so multi-word names such as fertilizer brands survive.
2. **Clean.** Normalize Unicode (NFC, fold the nukta), strip annotation marks, drop non-farming and non-Devanagari junk. The 127 removals: run-on phrase 55, non-Devanagari 41, Latin script 21, annotation artifact 7, numeric 3. A further 161 rows fold into an existing entry, leaving 18,646.
3. **Tier and weight.** Give each entry a tier (core 2,771, long tail 13,015, phrase 2,860), a category (13; the largest are pest, disease or concern 4,672, practice 4,153, crop 1,785, fertilizer or chemical 1,068) and a weight (4: 9,195 terms; 3: 7,498; 2: 1,953).
4. **Phonemize.** Write each entry as the sounds it is spoken with, in one scheme shared across Indic scripts, dropping the silent final vowel and the rule-governed middle ones (§5.4.2). A quarter of entries, 4,756 of 18,646 in 1,957 groups, sound like another entry, which is why the matcher refuses a tie.
5. **Cluster.** For each of 2,563 headwords, collect the entries that sound close to it: the sets an ASR model could confuse.
6. **Adjudicate.** Label every pair as the same word, the same meaning, a different meaning, or unrelated, with a cue on each different-meaning pair. Of 13,904 pairs: same word 2,287, same meaning 647, different meaning 10,335, unrelated 635 (Table 5)^L.
7. **Family and over-merge guard.** Group same-meaning variants into 398 families comprising 950 member words, mean size 2.39, then recheck every merge: 118 members change meaning and are pulled back out, leaving 2,934 pairs safe to fold and 10,335 context-only pairs.

The pipeline uses two products: the 2,934 safe pairs, folded only when the ASR model produced a word the corpus has never seen (§5.4.2), and the 10,335 different-meaning pairs with their cues, flagged and never rewritten. Step 7 is what makes the list safe: without it, folding one variant family would have collapsed 118 distinct meanings, including the दवा (*davā*) and दावा (*dāvā*) case of Figure 5. Sound-alike chains join 2,631 of the 2,771 core words into one group, so the pairs are overlapping neighbourhoods, not clean partitions.

B Farming-Term and Per-Language Detail

Both views here repeat under a second metric what §7.1 already shows under WER.

- **Selection loses a few farming words and gains more precision.** At S3 recall falls slightly, precision rises, and F1 rises on all four ASR models, sharply so on multi-speaker clips (Table 19). The precision gain is the bystander’s farming words leaving the transcript; the recall loss is the farmer’s own words clipped at turn edges, the harm corpus WER hides and why selection is gated (§7.4).
- **Repair raises farming-term recall on every ASR model.** S3 to S4 moves recall up on all four, the stage doing the job it was built for, while its effect on corpus WER is at most a thousandth: it shows its worth on the metric it targets, not the one averaging over every word.
- **What F1 adds to WER.** WER charges a lost crop name the price of a lost function word, and reads a removed bystander the same as a removed farmer. F1 and the farming-term rate separate those cases.

Table 19: Farming-term recall / precision / F1 across the ladder, Hindi. Cells are recall / precision / F1.

Subset	ASR model	S1	S3	S4
All Hindi ($n = 1,120$)	Gemini 3.7 Flash	0.881 / 0.809 / 0.831	0.856 / 0.832 / 0.836	0.857 / 0.831 / 0.837
	Sarvam saaras:v3	0.832 / 0.826 / 0.816	0.824 / 0.859 / 0.831	0.825 / 0.856 / 0.830
	Azure Speech	0.829 / 0.800 / 0.801	0.818 / 0.837 / 0.818	0.819 / 0.835 / 0.817
	IndicConformer 600M	0.811 / 0.832 / 0.809	0.803 / 0.839 / 0.812	0.805 / 0.834 / 0.811
Multi-speaker ($n = 353$)	Gemini 3.7 Flash	0.870 / 0.706 / 0.755	0.805 / 0.764 / 0.772	0.807 / 0.762 / 0.772
	Sarvam saaras:v3	0.801 / 0.734 / 0.744	0.777 / 0.809 / 0.779	0.778 / 0.804 / 0.776
	Azure Speech	0.805 / 0.708 / 0.734	0.782 / 0.790 / 0.771	0.783 / 0.787 / 0.770
	IndicConformer 600M	0.769 / 0.768 / 0.752	0.759 / 0.797 / 0.767	0.761 / 0.792 / 0.765

Repair works off a Devanagari word list and is gated on the script of each word, so Hindi carries almost all of what it moves: Telugu passes through untouched, and Odia shifts only where an ASR model wrote Devanagari (Table 20). Selection carries the gain in all three languages.

Table 20: Per-language ladder, Gemini 3.7 Flash. Repair edits Devanagari, so it also touches Odia. Best per row in green bold.

Language	n	S1	S2	S3	S4
		WER / F1	WER / F1	WER / F1	WER / F1
Hindi	1,126	0.395 / 0.772	0.393 / 0.770	0.266 / 0.787	0.266 / 0.788
Telugu	996	0.460 / 0.651	0.457 / 0.650	0.394 / 0.647	0.394 / 0.647
Odia	551	0.540 / 0.622	0.532 / 0.621	0.459 / 0.616	0.461 / 0.617